ORBITAL_X PROJECT

SYSTEM ARCHITECTURE, ROOT ORCHESTRATION, AND AI MICROSERVICE

TABLE OF CONTENTS

1. ROOT WORKSPACE FILES & SCRIPTS

1.1 start.bat

1.2 stop.bat

1.3 package.json (Root Workspace)

1.4 docker-compose.yml

1.5 .gitignore

2. AI MICROSERVICE & SATELLITE ANOMALY DIAGNOSTIC ENGINE (AI/)

2.1 AI Architecture & Mathematical Theory (Gaussian Naive Bayes)

2.2 ai_inference_service.py (Port 5050 HTTP Diagnostic Microservice)

2.3 train_classifier.py (Offline Model Training Pipeline)

2.4 test_ai_diagnosis.py (Aerospace Emergency Scenario Validation Suite)

2.5 generate_dataset.py (Telemetry Anomaly Dataset Generator)

2.6 fine_tune_llm.py (Prompt Generation for LLM Fine-Tuning)

2.7 generate_pdf_guide.py & generate_beginner_pdf.py (Automated PDF Report Generators)

2.8 Dataset Schema & Failure Modes (satellite_telemetry_anomaly_dataset.csv)

ROOT WORKSPACE FILES & SCRIPTS

1.1 start.bat (Root Launcher Script)

PURPOSE:

Provides a Windows batch script that starts all three microservices in independent terminal windows

and opens the mission control dashboard in the user's default browser.

DETAILED BREAKDOWN:

- Line 1-6: Disables command echoing (`@echo off`), sets the console title to "OrbitalX Mission Control Launcher",

and prints the header banner.

- Line 8-9: Starts the AI Diagnostic microservice:

start "OrbitalX AI Microservice" cmd /k "cd /d %~dp0AI && python ai_inference_service.py --serve"

`%~dp0` expands to the absolute drive and directory of the batch file itself. `cmd /k` keeps the terminal

window open after starting so that operators can inspect live AI classification logs.

- Line 11-12: Launches the Express backend:

start "OrbitalX Backend" cmd /k "cd /d %~dp0OrbitalX\backend && npm run dev"

This boots nodemon on port 4000, connects to MongoDB (or launches in-memory fallback), seeds data, and

starts the continuous 10-second telemetry physics simulation loop.

- Line 14-15: Launches the Vite frontend development web server:

start "OrbitalX Frontend" cmd /k "cd /d %~dp0OrbitalX\frontend && npm run dev"

Boots the React 19 single-page application on `http://localhost:4173`.

- Line 17-19: Uses `timeout /t 5 /nobreak > nul` to pause for 5 seconds, giving backend database connections

and the AI microservice socket bindings time to initialize.

- Line 21-22: Launches the user's web browser to `http://localhost:4173/dashboard`.

- Line 24-32: Displays the active service endpoint summary and pauses.

1.2 stop.bat (Graceful System Shutdown)

PURPOSE:

Provides a quick mechanism to terminate all active node, python, and vite child processes spawned

by start.bat without needing to hunt down PIDs in Windows Task Manager.

DETAILED BREAKDOWN:

- Calls `taskkill /F /IM node.exe /T` and `taskkill /F /IM python.exe /T` to forcefully terminate

  the child processes across port 4000, port 4173, and port 5050.

- Prints confirmation to the operator that all aerospace processes have returned to idle.

1.3 package.json (Root Workspace Manifest)

PURPOSE:

Configures the monorepo workspace scripts, enabling operators to control individual subsystems

from the root directory using standard npm commands.

KEY SCRIPTS:

- "start:backend": "npm --prefix OrbitalX/backend run dev"

  Invokes the backend nodemon runner inside the backend subfolder.

- "start:frontend": "npm --prefix OrbitalX/frontend run dev"

  Invokes Vite dev server inside the frontend subfolder.

- "start:ai": "python AI/ai_inference_service.py --serve"

  Executes the HTTP server for the AI subsystem.

- "test:ai": "python AI/test_ai_diagnosis.py"

  Runs the 5 automated aerospace failure tests.

- "build": "npm --prefix OrbitalX/frontend run build"

Triggers Rolldown/Vite bundler to produce production assets in `dist/`.

- "test:security": "node OrbitalX/backend/tests/securityAudit.js"

Runs the 13 automated security tests validating RBAC and NoSQL injection defenses.

- "docker:up": "docker-compose up --build -d"

- "docker:down": "docker-compose down"

1.4 docker-compose.yml

PURPOSE:

Defines containerized multi-service deployment for running OrbitalX in staging/production environments.

SERVICES CONFIGURED:

1. `mongodb`: Uses official `mongo:7.0` image, mounts `mongo_data` volume for persistent storage,

exposes port 27017, and runs health checks.

2. `backend`: Builds from `./OrbitalX/backend/Dockerfile`, injects environment variables

(`PORT=4000`, `MONGODB_URI=mongodb://mongodb:27017/orbitalx`, `JWT_SECRET`), links to `mongodb`,

exposes port 4000.

3. `frontend`: Builds from `./OrbitalX/frontend/Dockerfile`, uses multi-stage build (Node build -> Nginx Alpine),

maps port 80 to host port 80/4173, reverse proxies `/api` calls to the backend container.

AI MICROSERVICE & SATELLITE ANOMALY DIAGNOSTIC ENGINE (AI/)

2.1 AI Architecture & Mathematical Theory (Gaussian Naive Bayes)

MOTIVATION:

In aerospace mission control, flight software and edge microservices cannot rely on heavy external

frameworks (like PyTorch or TensorFlow) that take minutes to load, require gigabytes of RAM, or depend

on dynamic C-libraries that may fail to compile in radiation-hardened or isolated ground computers.

OrbitalX implements a zero-external-dependency, pure-Python Gaussian Naive Bayes classifier.

MATHEMATICAL FORMULATION:

Given a telemetry feature vector X = [x_1, x_2, ..., x_16] consisting of 16 physical telemetry channels:

1. battery_temp_k (Kelvin)

2. battery_voltage_v (Volts)
3. battery_current_a (Amperes)
4. bus_5v_rail (Volts)
5. gyro_rate_deg_s (Degrees/sec)
6. reaction_wheel_rpm (RPM)
7. wheel_current_a (Amperes)
8. rf_vswr (Voltage Standing Wave Ratio)
9. rf_power_w (Watts)
10. cpu_temp_k (Kelvin)
11. cpu_load_pct (Percent)
12. memory_ecc_errors (Counter)
13. i2c_bus_ack_pct (Percent)
14. chassis_accelerometer_g (G-force)
15. hull_pressure_kpa (Kilopascals)
16. lidar_range_m (Meters)

The model computes the posterior probability for each failure class C_k:

P(C_k | X) proportional to P(C_k) * Product[P(x_j | C_k)]

Where:

- P(C_k) is the class prior probability = (Count of class C_k samples) / (Total samples)
- P(x_j | C_k) follows the Gaussian probability density function:

P(x_j | C_k) = (1 / (sqrt(2*pi) * sigma_{k,j})) * exp(- (x_j - mu_{k,j})^2 / (2 * sigma_{k,j}^2))

To prevent numerical underflow with small floating point probabilities, computation is executed in the

log-domain:

log P(C_k | X) = log P(C_k) + Sum[-log(sigma_{k,j} * sqrt(2*pi)) - ((x_j - mu_{k,j})^2 / (2 * sigma_{k,j}^2))]

The predicted failure class is:

C* = argmax_{C_k} log P(C_k | X)

Inference time is under 0.8 milliseconds, enabling real-time telemetry stream evaluation.

2.2 AI/ai_inference_service.py (Port 5050 HTTP Server)

PURPOSE:

Acts as a standalone HTTP microservice listening on port 5050. Receives incoming JSON payloads

containing satellite telemetry, runs the classifier, maps the result to emergency root-cause analysis

and executable telecommands, and returns a JSON response.

KEY COMPONENTS:

1. `AerospaceUnpickler(pickle.Unpickler)`:

A custom pickle unpickler subclass that dynamically resolves the `AerospaceNaiveBayesClassifier` class

when loading `satellite_classifier_model.pkl`, guaranteeing clean deserialization regardless of

invocation namespace.

2. Auto-training Fallback (Lines 28-30):

Checks if `satellite_classifier_model.pkl` exists. If absent, it automatically triggers `train_classifier.main()`

to generate the model file from `satellite_telemetry_anomaly_dataset.csv`.

3. `diagnose_telemetry(telemetry_dict)`:

- Extracts the 16 feature values from the input dictionary.

- Provides safe aerospace nominal defaults (e.g., `battery_temp_k=298.0`, `hull_pressure_kpa=101.3`,

`lidar_range_m=80000.0`) if telemetry keys are omitted.

- Calls `CLASSIFIER.predict_one(vec)`.

- Returns structured diagnosis metadata including:

- `predicted_failure_mode`: Human-readable identifier (e.g. `BATTERY_CELL_THERMAL_RUNAWAY`)

- `anomaly_category`: `INNER_ERROR` vs `OUTER_COLLISION_DAMAGE`

- `subsystem`: `EPS`, `ADCS`, `COMMS`, `TCS`, `OBC`, `ENV_STRUCTURE`, `PAYLOAD`

- `recommended_telecommand`: The primary flight mitigation command (e.g. `CMD_EPS_SAFE_MODE`)

- `root_cause_analysis`: Physical explanation of the electrical/mechanical fault

- `sop_checklist`: Step-by-step Standard Operating Procedure for operators

- `confidence_score`: Statistical probability score (e.g. 0.982)

4. `DiagnosticServerHandler(http.server.BaseHTTPRequestHandler)`:

- `do_OPTIONS()`: Returns CORS headers (`Access-Control-Allow-Origin: *`, `Allow-Methods: POST, GET, OPTIONS`),

allowing frontend direct queries.

- `do_GET()`:

- `/health`: Returns `{"status": "online", "model": "OrbitalX-Aerospace-NB-v1"}` with HTTP 200.

- `/diagnose`: Returns documentation on how to POST telemetry.

- `do_POST()`:

- Reads `Content-Length` bytes from `self.rfile`.

- Deserializes JSON telemetry payload.

- Calls `diagnose_telemetry(data)`.

- Serializes response to UTF-8 JSON with HTTP 200.

2.3 AI/train_classifier.py (Model Training Pipeline)

PURPOSE:

Trains the Gaussian Naive Bayes classifier on 10,000+ synthetic aerospace anomaly telemetry rows

from `satellite_telemetry_anomaly_dataset.csv` and serializes the parameters into `satellite_classifier_model.pkl`.

DETAILED FLOW:

1. `load_data()`: Reads the CSV dataset using Python's `csv.DictReader`. Extracts the 16 numerical features

into feature vectors X, target `failure_mode` labels into y, and metadata dictionaries

(category, subsystem, severity, telecommand, root_cause, sop) into `metadata`.

2. `fit(X, y, metadata)`:

- Determines unique failure classes.

- Calculates prior probabilities `class_priors[c] = count / total`.

- For every feature j in class c, computes mean and sample variance. If variance is near zero,

enforces a minimum standard deviation floor of 1e-3 to avoid division by zero.

- Links each class to its first encountered operational SOP and mitigation command.

3. Serializes the trained instance to `satellite_classifier_model.pkl` via `pickle.dump()`.

2.4 AI/test_ai_diagnosis.py (Automated Validation Suite)

PURPOSE:

Simulates 5 critical mission emergencies and asserts that the classifier correctly diagnoses the domain,

subsystem, failure mode, and 1-click telecommand:

- Scenario 1: Space Debris Conjunction Threat (LIDAR range 112m closing fast -> `CMD_PROP_ARM_CAM_BURN`)

- Scenario 2: Battery Thermal Runaway (Battery temp 339K -> `CMD_EPS_SAFE_MODE`)

- Scenario 3: Reaction Wheel Bearing Seizure (Wheel RPM stalls to 46 -> `CMD_ADCS_DESPIN_WHEELS`)

- Scenario 4: Micrometeoroid Kinetic Impact (11.2G shock -> `CMD_EPS_SAFE_MODE`)

- Scenario 5: Radiation Single Event Upset (414 ECC errors -> `CMD_OBC_MEM_SCRUB_ENABLE`)

Includes a `--benchmark` mode to evaluate accuracy over 100 randomly sampled telemetry vectors.

2.5 AI/generate_dataset.py (Telemetry Dataset Generator)

PURPOSE:

Generates the 10,000-row `satellite_telemetry_anomaly_dataset.csv` using realistic physical formulas

and gaussian noise models for satellite subsystems in Low Earth Orbit (LEO) and Geostationary Orbit (GEO).

Injects realistic solar flux (1367 W/m2 variation), South Atlantic Anomaly (SAA) radiation transit,

and orbital velocity physics.

2.6 AI/fine_tune_llm.py (Fine-Tuning Prompt Generator)

PURPOSE:

Converts the structured CSV telemetry and failure records into JSONL prompt-completion pairs

(`ai_satellite_training_prompts.jsonl`) suitable for fine-tuning Large Language Models (OpenAI GPT-4o,

Google Gemini 1.5/2.0, Meta LLaMA 3, Mistral) for aerospace incident copilots.

2.7 AI/generate_pdf_guide.py & generate_beginner_pdf.py

PURPOSE:

Automated report generators that compile high-resolution technical and mentor documentation into:

- `OrbitalX_AI_Anomaly_and_Collision_Defense_Guide.pdf`

- `OrbitalX_AI_Beginners_Guide.pdf`

Outlines aerospace defense mechanisms, orbital mechanics, and operator training workflows.